

Лабораторная работа № 6. Настройка пропускной способности глобальной сети с помощью Token Bucket Filter

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	5
2	Предварительные сведения	6
3	Задание	9
4	Выполнение лабораторной работы	10
4.1	Запуск лабораторной топологии	10
4.2	Интерактивные эксперименты	13
4.2.1	Ограничение скорости на конечных хостах	13
4.2.2	Ограничение скорости на коммутаторах	15
4.2.3	Объединение NETEM и TBF	17
4.3	Воспроизводимые эксперименты	20
5	Выводы	24

Список иллюстраций

4.1	Запуск виртуальной машины	10
4.2	Исправление прав запуска X-соединения	11
4.3	Задание топологии	11
4.4	Информация на коммутаторах s1 и s2	11
4.5	Информация на хостах h1 и h2	12
4.6	Проверка подключения между хостами	12
4.7	Запуск iPerf3 на обоих хостах в разных режимах на обоих хостах .	12
4.8	Установка пропускной способности на 10 Гбит/с на интерфейсе h1-eth0	13
4.9	Извлечение тактовой частоты	14
4.10	Запуск iPerf3 на обоих хостах в разных режимах на обоих хостах .	14
4.11	Удаление модифицированной конфигурации на хосте h1	15
4.12	Применение правила ограничения скорости	16
4.13	Запуск iPerf3 на обоих хостах в разных режимах на обоих хостах .	16
4.14	Удаление модифицированной конфигурации на s1	17
4.15	Объединение NETEM и TBF	18
4.16	Проверка соединения от хоста h1 к хосту h2	18
4.17	Добавление второго правила	19
4.18	Запуск iPerf3 на обоих хостах в разных режимах на обоих хостах .	19
4.19	Удаление модифицированной конфигурации на s1	20
4.20	Создание каталога	20
4.21	Скрипт lab_netem_iii.py	21
4.22	Создание Makefile	21
4.23	Создание ping_plot	22
4.24	Запуск скрипта и получение файлов	22
4.25	Запуск geeqie	22
4.26	Финальный график	23

Список таблиц

1 Цель работы

Основной целью работы является знакомство с принципами работы дисциплины очереди Token Bucket Filter, которая формирует входящий/исходящий трафик для ограничения пропускной способности, а также получение навыков моделирования и исследования поведения трафика посредством проведения интерактивного и воспроизводимого экспериментов в Mininet

2 Предварительные сведения

Token Bucket Filter (TBF) представляет собой алгоритм, используемый в сетях с коммутацией пакетов для ограничения пропускной способности и пиковой нагрузки трафика. Передача поступающих в очередь системы (queue) пакетов данных осуществляется при наличии в специальном буфере (bucket) необходимого числа разрешений на передачу (или токенов). Токены могут быть представлены в виде пакетов или числа байтов, поступающих в буфер (корзину) фиксированного размера с фиксированной скоростью.

Максимальная средняя скорость отправки потока данных из очереди системы зависит от скорости прибытия в специализированный буфер разрешений на передачу N единиц данных. Очередной пакет может быть отправлен только при получении числа разрешений, достаточного для передачи данных, объём которых больше или равен размеру пакета. Если разрешений на передачу достаточно, то необходимое число токенов удаляется из специализированного буфера, а пакет данных отправляется. Если пакет поступит в очередь системы и не будет располагать необходимым количеством разрешений, то токены не удаляются из специализированного буфера, а сам пакет данных может быть отброшен, поставлен в очередь или передан, но помечен как несоответствующий условиям передачи.

Дисциплина TBF реализована в виде буфера (queue), постоянно заполняющегося токенами с заданной скоростью. Наиболее важным параметром буфера является его размер, определяющий количество хранимых токенов.

Если данные прибывают со скоростью, равной скорости входящих токенов, то

каждый пакет имеет соответствующий токен и проходит очередь без задержки. Если данные прибывают со скоростью, меньшей скорости поступления токенов, то лишь часть существующих токенов будет уничтожаться, поэтому они станут накапливаться до размера специализированного буфера. Далее накопленные токены могут использоваться при всплесках, для передачи данных со скоростью, превышающей скорость пребывающих токенов.

Если данные прибывают быстрее, чем токены, то в буфере со временем не останется токенов, что заставит дисциплину приостановить передачу данных. Эта ситуация называется «превышением». Если пакеты продолжают поступать, то токены начинают уничтожаться. Данная ситуация позволяет административно ограничивать доступную полосу пропускания.

Накопленные токены позволяют пропускать короткие всплески, но при продолжительном превышении пакеты будут задерживаться, а в крайнем случае — уничтожаться.

Алгоритм TBF обладает свойством взрывоопасности (burstiness), когда корзина полностью заполнена (т. е. никакие пакеты не потребляют токены) и новые пакеты будут потреблять токены сразу, без ограничений. Всплеск определяется как количество токенов, которые могут поместиться в корзину, или как размер (ёмкость) корзины. Для обеспечения ограничения и контроля над всплесками при поступлении пакетов формируется ещё одна корзина, с размером, равным максимально передаваемому элементу данных (Maximum Transmission Unit, MTU). Скорость обработки этого буфера намного превышает исходную (peak rate — пиковая скорость).

Основной синтаксис `tbft`, используемый с `tc` в Linux:

```
tc qdisc [add | ...] dev [dev_id] root tbft limit [BYTES]
burst [BYTES] rate [BPS] [mtu BYTES] [ peakrate BPS ] [
latency TIME ]
```

Здесь:

- tc: инструмент управления трафиком Linux;
- qdisc: дисциплина очереди (qdisc), представляющая собой набор правил, определяющих порядок, в котором обслуживаются пакеты, поступающие с выходных данных IP-протокола;
-
- dev [dev_id]: задаёт интерфейс;
- tbf: указывает, что используется алгоритм Token Bucket Filter;
- limit [BYTES]: размер очереди пакетов в байтах;
- burst [BYTES]: количество байтов, которое может поместиться в корзину;
- rate [BPS]: скорость передачи данных, определяемая частотой, с которой токены добавляются в корзину;
- mtu [BYTES]: максимальная единица передачи в байтах;
- peakrate [BPS] (пиковая скорость [бит/с]): максимальная скорость передачи пакета;
- latency [TIME]: максимальное время ожидания пакета в очереди

3 Задание

1. Задайте топологию, состоящую из двух хостов и двух коммутаторов с назначенной по умолчанию mininet сетью 10.0.0.0/8.
2. Проведите интерактивные эксперименты по ограничению пропускной способности сети с помощью TBF в эмулируемой глобальной сети.
3. Самостоятельно реализуйте воспроизводимые эксперимент по применению TBF для ограничения пропускной способности. Постройте соответствующие графики.

4 Выполнение лабораторной работы

4.1 Запуск лабораторной топологии

Запускаем виртуальную машину. Здесь ждет легкая неожиданность, но это не помешало выполнить лабораторную работу (рис. 4.1):

```
mininet@192.168.56.101's password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-42-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Last login: Sat Nov  8 08:11:44 2025 from 192.168.56.1
mininet@mininet-vm:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.101 netmask 255.255.255.0 broadcast 192.168.56.255
    ether 08:00:27:40:a4:3f txqueuelen 1000 (Ethernet)
    RX packets 117 bytes 18812 (18.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 53 bytes 8991 (8.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    ether 08:00:27:19:7d:07 txqueuelen 1000 (Ethernet)
    RX packets 374 bytes 45027 (45.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 394 bytes 35962 (35.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 56 bytes 4984 (4.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 56 bytes 4984 (4.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet@mininet-vm:~$
```

Рис. 4.1: Запуск виртуальной машины

В виртуальной машине mininet при необходимости исправим права запуска X-соединения. Скопируем значение куки (MIT magic cookie) своего пользователя mininet в файл для пользователя root(рис. 4.2):

```

mininet@mininet-vm:~$ xauth list $DISPLAY
mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 c513ec3fcc72804731e9f74fa948aa4e
mininet@mininet-vm:~$ sudo -i
root@mininet-vm:~# xauth add mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 c513ec3fcc72804731e9f74fa948aa4e
root@mininet-vm:~# logout
mininet@mininet-vm:~$

```

Рис. 4.2: Исправление прав запуска X-соединения

После выполнения этих действий графические приложения должны запускаться под пользователем mininet.

Зададим топологию сети, состоящую из двух хостов и двух коммутаторов с назначенной по умолчанию mininet сетью 10.0.0.0/8 (рис. 4.3):



Рис. 4.3: Задание топологии

После введения этой команды запустятся терминалы двух хостов, двух коммутаторов и контроллера.

На хостах h1, h2 и на коммутаторах s1, s2 введите команду `ifconfig`, чтобы отобразить информацию, относящуюся к их сетевым интерфейсам и назначенным им IP-адресам. В дальнейшем при работе с NETEM и командой `tc` будут использоваться интерфейсы h1-eth0, h2-eth0, s1-eth2 (рис. 4.4):

```

switch: s1 (root@mininet-vm)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
s1-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ae:0f:60:92:b7:8f txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
s2-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ca:22:34:fa:7d:85 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
s2-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether de:79:bc:15:83:4e txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
root@mininet-vm:/home/mininet#

switch: s1 (root@mininet-vm)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
s1-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ae:0f:60:92:b7:8f txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
s2-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ca:22:34:fa:7d:85 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
s2-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether de:79:bc:15:83:4e txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
root@mininet-vm:/home/mininet#

```

Рис. 4.4: Информация на коммутаторах s1 и s2

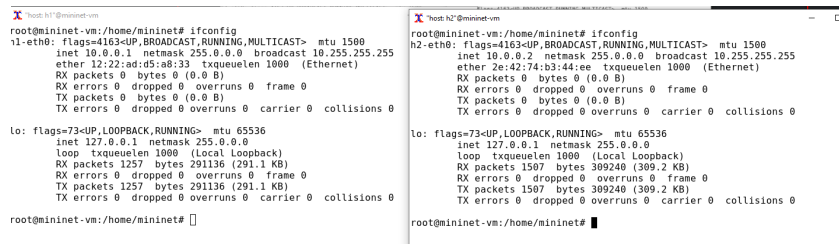


Рис. 4.5: Информация на хостах h1 и h2

Проверим подключение между хостами h1 и h2 с помощью команды ping с параметром -c 4 (рис. 4.6):

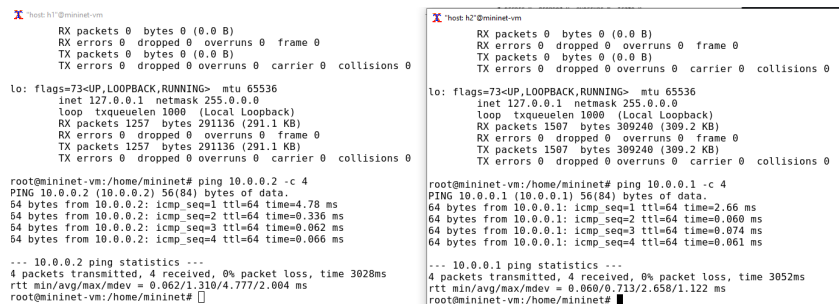


Рис. 4.6: Проверка подключения между хостами

В терминале хоста h2 запустим iPerf3 в режиме сервера и в терминале хоста h1 запустим iPerf3 в режиме клиента (рис. 4.7):

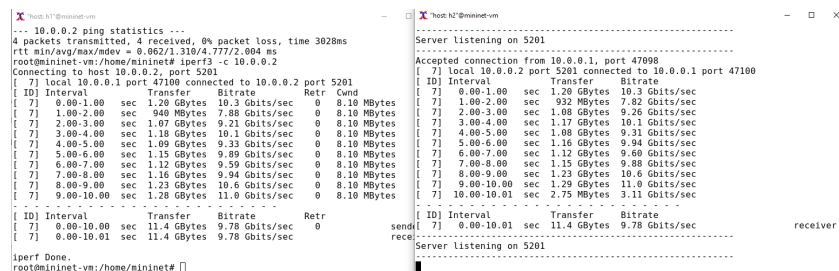


Рис. 4.7: Запуск iPerf3 на обоих хостах в разных режимах на обоих хостах

После завершения работы iPerf3 на хосте h1 остановим iPerf3 на хосте h2, нажав Ctrl + c.

4.2 Интерактивные эксперименты

4.2.1 Ограничение скорости на конечных хостах

Команду `tc` можно применить к сетевому интерфейсу устройства для формирования исходящего трафика. Требуется ограничить скорость отправки данных с конечного хоста с помощью фильтра Token Bucket Filter (tbf)

Изменим пропускную способность хоста `h1`, установив пропускную способность на 10 Гбит/с на интерфейсе `h1-eth0` и параметры TBF-фильтра(рис. 4.8):

```
host: h1@mininet-vm
rtt min/avg/max/mdev = 0.062/1.310/4.777/2.004 ms
root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
[ 7] local 10.0.0.1 port 47100 connected to 10.0.0.2 port 5201
[ ID] Interval      Transfer    Bitrate      Retr  Cwnd
[ 7]  0.00-1.00  sec    1.20 GBytes  10.3 Gbits/sec    0   8.10 MBytes
[ 7]  1.00-2.00  sec    0.94 GBytes  7.88 Gbits/sec    0   8.10 MBytes
[ 7]  2.00-3.00  sec    1.07 GBytes  9.21 Gbits/sec    0   8.10 MBytes
[ 7]  3.00-4.00  sec    1.18 GBytes  10.1 Gbits/sec    0   8.10 MBytes
[ 7]  4.00-5.00  sec    1.09 GBytes  9.33 Gbits/sec    0   8.10 MBytes
[ 7]  5.00-6.00  sec    1.15 GBytes  9.89 Gbits/sec    0   8.10 MBytes
[ 7]  6.00-7.00  sec    1.12 GBytes  9.59 Gbits/sec    0   8.10 MBytes
[ 7]  7.00-8.00  sec    1.16 GBytes  9.94 Gbits/sec    0   8.10 MBytes
[ 7]  8.00-9.00  sec    1.23 GBytes  10.6 Gbits/sec    0   8.10 MBytes
[ 7]  9.00-10.00 sec    1.28 GBytes  11.0 Gbits/sec    0   8.10 MBytes
[ ID] Interval      Transfer    Bitrate      Retr
[ 7]  0.00-10.00  sec    11.4 GBytes  9.78 Gbits/sec    0
[ 7]  0.00-10.01  sec    11.4 GBytes  9.78 Gbits/sec    0
sender
receiver

iperf Done.
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root tbf rate 10gb
it burst 5000000 limit 15000000
root@mininet-vm:/home/mininet#
```

Рис. 4.8: Установка пропускной способности на 10 Гбит/с на интерфейсе `h1-eth0`

Здесь:

- `sudo`: включить выполнение команды с более высокими привилегиями безопасности;
- `tc`: вызвать управление трафиком Linux;
- `qdisc`: изменить дисциплину очередей сетевого планировщика;
- `add` (добавить): создать новое правило;
- `dev h1-eth0 root`: интерфейс, на котором будет применяться правило;

- **tbft**: использовать алгоритм Token Bucket Filter;
- **rate**: указать скорость передачи (10 Гбит/с);
- **burst**: количество байтов, которое может поместиться в корзину (5 000 000);
- **limit**: размер очереди в байтах (15 000 000).

Фильтр **tbft** требует установки значения всплеска при ограничении скорости. Это значение должно быть достаточно высоким, чтобы обеспечить установленную скорость. Она должна быть не ниже указанной частоты, делённой на **NZ**, где **NZ** — тактовая частота, настроенная как параметр ядра, и может быть извлечена с помощью следующей команды (рис. 4.9):

```
root@mininet-vm:/home/mininet# egrep '^CONFIG_HZ_[0-9]+' /boot/config-`uname -r`
CONFIG_HZ_250=y
root@mininet-vm:/home/mininet#
```

Рис. 4.9: Извлечение тактовой частоты

Для расчёта значения всплеска (**burst**) необходимо скорость передачи (10 Гбит/с или 10 Gbps = 10,000,000,000 bps) разделить на полученное таким образом значение **NZ** (на хосте **h1** **NZ** = 250): $\text{Burst} = 10,000,000,000 / 250 = 40,000,000 \text{ bits} = 40,000,000 / 8 \text{ bytes} = 5,000,000 \text{ bytes}$.

С помощью **iPerf3** проверим, что значение пропускной способности изменилось. В терминале хоста **h2** запустим **iPerf3** в режиме сервера и в терминале хоста **h1** запустим **iPerf3** в режиме клиента (рис. 4.10):

```

root@mininet-vm:/home/mininet# egrep '^CONFIG_HZ_[0-9]+' /boot/config-`uname -r`
CONFIG_HZ_250=y
root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
[ ID] Interval      Transfer     Bitrate      Retr  Cwnd
[ 0] 0.00-1.00 sec  1.09 GBytes  9.36 Gbits/sec  0    8.23 MBytes
[ 1] 1.00-2.00 sec  1.11 GBytes  9.56 Gbits/sec  0    8.23 MBytes
[ 2] 2.00-3.00 sec  1.10 GBytes  9.49 Gbits/sec  0    8.23 MBytes
[ 3] 3.00-4.00 sec  1.11 GBytes  9.53 Gbits/sec  0    8.23 MBytes
[ 4] 4.00-5.00 sec  1.11 GBytes  9.54 Gbits/sec  0    8.23 MBytes
[ 5] 5.00-6.00 sec  1.12 GBytes  9.61 Gbits/sec  0    8.23 MBytes
[ 6] 6.00-7.00 sec  1.10 GBytes  9.49 Gbits/sec  0    8.23 MBytes
[ 7] 7.00-8.00 sec  1.09 GBytes  9.40 Gbits/sec  0    8.23 MBytes
[ 8] 8.00-9.00 sec  1.12 GBytes  9.58 Gbits/sec  0    8.23 MBytes
[ 9] 9.00-10.00 sec 1.11 GBytes  9.50 Gbits/sec  0    8.23 MBytes
[ ID] Interval      Transfer     Bitrate      Retr  Cwnd
[ 0] 0.00-10.00 sec 11.1 GBytes  9.51 Gbits/sec  0
[ 1] 0.00-10.02 sec 11.1 GBytes  9.49 Gbits/sec  0
iperf Done.
root@mininet-vm:/home/mininet#

Server listening on 5201
Accepted connection from 10.0.0.1, port 47102
[ 0] local 10.0.0.2 port 5201 connected to 10.0.0.1 port 47104
[ ID] Interval      Transfer     Bitrate      Retr  Cwnd
[ 0] 0.00-1.00 sec  1.08 GBytes  9.26 Gbits/sec
[ 1] 1.00-2.01 sec  1.11 GBytes  9.48 Gbits/sec
[ 2] 2.01-3.01 sec  1.10 GBytes  9.40 Gbits/sec
[ 3] 3.01-4.00 sec  1.11 GBytes  9.60 Gbits/sec
[ 4] 4.00-5.00 sec  1.10 GBytes  9.48 Gbits/sec
[ 5] 5.00-6.00 sec  1.11 GBytes  9.52 Gbits/sec
[ 6] 6.00-7.01 sec  1.12 GBytes  9.60 Gbits/sec
[ 7] 7.01-8.00 sec  1.10 GBytes  9.51 Gbits/sec
[ 8] 8.00-9.00 sec  1.10 GBytes  9.41 Gbits/sec
[ 9] 9.00-10.00 sec 1.11 GBytes  9.59 Gbits/sec
[10] 10.00-10.02 sec 18.3 MBytes  9.17 Gbits/sec
[ ID] Interval      Transfer     Bitrate
[ 0] 0.00-10.02 sec 11.1 GBytes  9.49 Gbits/sec
Server listening on 5201

```

Рис. 4.10: Запуск **iPerf3** на обоих хостах в разных режимах на обоих хостах

После завершения работы **iPerf3** на хосте **h1** остановим **iPerf3** на хосте **h2**, нажав **Ctrl + c**.

Удалим модифицированную конфигурацию на хосте h1 (рис. 4.11):

```

root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
7] local 10.0.0.1 port 47104 connected to 10.0.0.2 port 5201
ID] Interval      Transfer    Bitrate      Retr  Cwnd
7]  0.00-1.00    sec  1.09 GBytes  9.36 Gbits/sec    0   8.23 MBytes
7]  1.00-2.00    sec  1.11 GBytes  9.56 Gbits/sec    0   8.23 MBytes
7]  2.00-3.00    sec  1.10 GBytes  9.49 Gbits/sec    0   8.23 MBytes
7]  3.00-4.00    sec  1.11 GBytes  9.53 Gbits/sec    0   8.23 MBytes
7]  4.00-5.00    sec  1.11 GBytes  9.54 Gbits/sec    0   8.23 MBytes
7]  5.00-6.00    sec  1.12 GBytes  9.61 Gbits/sec    0   8.23 MBytes
7]  6.00-7.00    sec  1.10 GBytes  9.49 Gbits/sec    0   8.23 MBytes
7]  7.00-8.00    sec  1.09 GBytes  9.40 Gbits/sec    0   8.23 MBytes
7]  8.00-9.00    sec  1.12 GBytes  9.58 Gbits/sec    0   8.23 MBytes
7]  9.00-10.00   sec  1.11 GBytes  9.50 Gbits/sec    0   8.23 MBytes
- - - - -
ID] Interval      Transfer    Bitrate      Retr
7]  0.00-10.00   sec  11.1 GBytes  9.51 Gbits/sec    0
7]  0.00-10.02   sec  11.1 GBytes  9.49 Gbits/sec    0
sender
receiver

iperf Done.
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root
root@mininet-vm:/home/mininet#

```

Рис. 4.11: Удаление модифицированной конфигурации на хосте h1

4.2.2 Ограничение скорости на коммутаторах

При ограничении скорости на интерфейсе s1-eth2 коммутатора s1 все сеансы связи между коммутатором s1 и коммутатором s2 будут фильтроваться в соответствии с применяемыми правилами.

Применим правило ограничения скорости tbf с параметрами rate = 10gbit, burst = 5,000,000, limit= 15,000,000 к интерфейсу s1-eth2 коммутатора s1, который соединяет его с коммутатором s2 (рис. 4.12):

```

switch: s1" (root)@mininet-vm
s1-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ae:0f:60:92:b7:8f txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ca:22:34:fa:7d:85 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether de:79:bc:15:83:4e txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root tbf rate 10gb
it burst 5000000 limit 15000000
root@mininet-vm:/home/mininet#

```

Рис. 4.12: Применение правила ограничения скорости

Проверим конфигурацию с помощью инструмента `iperf3` для измерения пропускной способности. В терминале хоста `h2` запустим `iPerf3` в режиме сервера и в терминале хоста `h1` запустим `iPerf3` в режиме клиента (рис. 4.13):

```

iperf Done.
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root
root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
[ 7] local 10.0.0.1 port 47188 connected to 10.0.0.2 port 5201
[ ID] Interval      Transfer     Bitrate      Retr  Cwnd
[ 7] 0.00-1.00 sec  1.07 GBytes  9.28 Gbits/sec  1    8.19 MBytes
[ 7] 1.00-2.00 sec  1.11 GBytes  9.53 Gbits/sec  0    8.19 MBytes
[ 7] 2.00-3.00 sec  1.11 GBytes  9.52 Gbits/sec  0    8.19 MBytes
[ 7] 3.00-4.01 sec  1.11 GBytes  9.50 Gbits/sec  0    8.19 MBytes
[ 7] 4.01-5.00 sec  1.10 GBytes  9.53 Gbits/sec  0    8.19 MBytes
[ 7] 5.00-6.00 sec  1.11 GBytes  9.58 Gbits/sec  0    8.19 MBytes
[ 7] 6.00-7.00 sec  1.11 GBytes  9.56 Gbits/sec  0    8.19 MBytes
[ 7] 7.00-8.00 sec  1.11 GBytes  9.56 Gbits/sec  0    8.19 MBytes
[ 7] 8.00-9.00 sec  1.11 GBytes  9.56 Gbits/sec  0    8.19 MBytes
[ 7] 9.00-10.00 sec 1.10 GBytes  9.41 Gbits/sec  0    8.19 MBytes
[ ID] Interval      Transfer     Bitrate      Retr
[ 7] 0.00-10.00 sec 11.1 GBytes  9.49 Gbits/sec  1
[ 7] 0.00-10.02 sec 11.0 GBytes  9.47 Gbits/sec
iperf Done.
root@mininet-vm:/home/mininet#

```

```

root@h2" @mininet-vm
Server listening on 5201
Accepted connection from 10.0.0.1, port 47186
[ ID] Interval      Transfer     Bitrate      Retr  Cwnd
[ 7] 0.00-1.00 sec  1.06 GBytes  9.46 Gbits/sec  1    8.19 MBytes
[ 7] 1.00-2.00 sec  1.09 GBytes  9.39 Gbits/sec  0    8.19 MBytes
[ 7] 2.00-3.00 sec  1.12 GBytes  9.58 Gbits/sec  0    8.19 MBytes
[ 7] 3.00-4.00 sec  1.12 GBytes  9.52 Gbits/sec  0    8.19 MBytes
[ 7] 4.00-5.00 sec  1.11 GBytes  9.52 Gbits/sec  0    8.19 MBytes
[ 7] 5.00-6.00 sec  1.11 GBytes  9.50 Gbits/sec  0    8.19 MBytes
[ 7] 6.00-7.00 sec  1.11 GBytes  9.54 Gbits/sec  0    8.19 MBytes
[ 7] 7.00-8.00 sec  1.11 GBytes  9.51 Gbits/sec  0    8.19 MBytes
[ 7] 8.00-9.00 sec  1.11 GBytes  9.52 Gbits/sec  0    8.19 MBytes
[ 7] 9.00-10.00 sec 1.10 GBytes  9.47 Gbits/sec  0    8.19 MBytes
[ 7] 10.00-10.02 sec 256 KBytes  125 Mbits/sec
[ ID] Interval      Transfer     Bitrate
[ 7] 0.00-10.02 sec 11.0 GBytes  9.47 Gbits/sec
sender
receiver
Server listening on 5201

```

Рис. 4.13: Запуск `iPerf3` на обоих хостах в разных режимах на обоих хостах

После завершения работы `iPerf3` на хосте `h1` остановим `iPerf3` на хосте `h2`, нажав `Ctrl + c`.

Удалим модифицированную конфигурацию на коммутаторе `s1` (рис. 4.14):

```
"switch: s1" (root@mininet-vm)
ether ae:0f:60:92:b7:8f txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ca:22:34:fa:7d:85 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether de:79:bc:15:83:4e txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root tbf rate 10gb
it burst 5000000 limit 15000000
root@mininet-vm:/home/mininet# sudo tc qdisc del dev s1-eth2 root
root@mininet-vm:/home/mininet#
```

Рис. 4.14: Удаление модифицированной конфигурации на s1

4.2.3 Объединение NETEM и TBF

NETEM используется для изменения задержки, джиттера, повреждения пакетов и т.д. TBF может использоваться для ограничения скорости. Утилита tc позволяет комбинировать несколько модулей. При этом первая дисциплина очереди (qdisc1) присоединяется к корневой метке, последующие дисциплины очереди можно прикрепить к своим родителям, указав правильную метку.

Объединим NETEM и TBF, введя на интерфейсе s1-eth2 коммутатора s1 задержку, джиттер, повреждение пакетов и указав скорость (рис. 4.15):

```
"switch: s1" (root)@mininet-vm
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ca:22:34:fa:7d:85 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether de:79:bc:15:83:4e txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root tbf rate 10gb
it burst 5000000 limit 15000000
root@mininet-vm:/home/mininet# sudo tc qdisc del dev s1-eth2 root
root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root handle 1: net
em delay 10ms
root@mininet-vm:/home/mininet#
```

Рис. 4.15: Объединение NETEM и TBF

Здесь ключевое слово `handle` задаёт дескриптор подключения, имеющий смысл очередности подключения разных дисциплин `qdisc`.

Убедимся, что соединение от хоста `h1` к хосту `h2` имеет заданную задержку. Для этого запустим команду `ping` с параметром `-c 4` с терминала хоста `h1` (рис. 4.16):

```
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 4
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=13.6 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=11.2 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=10.9 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=10.3 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 10.280/11.508/13.649/1.281 ms
root@mininet-vm:/home/mininet#
```

Рис. 4.16: Проверка соединения от хоста `h1` к хосту `h2`

Добавим второе правило на коммутаторе `s1`, которое задаёт ограничение скорости с помощью `tbf` с параметрами `rate=2gbit`, `burst=1,000,000`, `limit=2,000,000` (рис. 4.17):

```

TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ca:22:34:fa:7d:85 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether de:79:bc:15:83:4e txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root tbf rate 10gb
it burst 5000000 limit 15000000
root@mininet-vm:/home/mininet# sudo tc qdisc del dev s1-eth2 root
root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root handle 1: net
em delay 10ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 parent 1: handle 2
: tbf rate 2gbit burst 1000000 limit 2000000
root@mininet-vm:/home/mininet#

```

Рис. 4.17: Добавление второго правила

Проверим конфигурацию с помощью инструмента `iperf3` для измерения пропускной способности. В терминале хоста `h2` запустим `iPerf3` в режиме сервера и в терминале хоста `h1` запустим `iPerf3` в режиме клиента (рис. 4.18):

```

root@h2:/home/mininet#
Server listening on 5201
Accepted connection from 10.0.0.1, port 47110
[ 7] Local 10.0.0.2 port 5201 connected to 10.0.0.1 port 47112
[ ID] Interval      Transfer     Bitrate
[ 7] 0.00-1.00 sec  204 MBytes  1.71 Gbits/sec
[ 7] 1.00-2.00 sec  228 MBytes  1.91 Gbits/sec
[ 7] 2.00-3.00 sec  226 MBytes  1.90 Gbits/sec
[ 7] 3.00-4.00 sec  227 MBytes  1.91 Gbits/sec
[ 7] 4.00-5.00 sec  185 MBytes  1.55 Gbits/sec
[ 7] 5.00-6.00 sec  187 MBytes  1.57 Gbits/sec
[ 7] 6.00-7.00 sec  191 MBytes  1.60 Gbits/sec
[ 7] 7.00-8.00 sec  200 MBytes  1.68 Gbits/sec
[ 7] 8.00-9.00 sec  204 MBytes  1.71 Gbits/sec
[ 7] 9.00-10.00 sec 204 MBytes  1.71 Gbits/sec
[ 7] 10.00-10.02 sec 1.31 MBytes  626 Mbits/sec
[ ID] Interval      Transfer     Bitrate
[ 7] 0.00-10.02 sec 2.01 GBytes  1.72 Gbits/sec
Server listening on 5201

root@h1:/home/mininet#
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3000ms
rtt min/avg/max/mdev = 10.280/11.508/13.649/1.281 ms
root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
[ 7] Local 10.0.0.1 port 47112 connected to 10.0.0.2 port 5201
[ ID] Interval      Transfer     Bitrate      Retr  Cwnd
[ 7] 0.00-1.00 sec  214 MBytes  1.80 Gbits/sec   90  3.16 MBytes
[ 7] 1.00-2.00 sec  229 MBytes  1.92 Gbits/sec    0  3.40 MBytes
[ 7] 2.00-3.00 sec  226 MBytes  1.90 Gbits/sec   180  2.52 MBytes
[ 7] 3.00-4.00 sec  228 MBytes  1.91 Gbits/sec    0  2.63 MBytes
[ 7] 4.00-5.00 sec  184 MBytes  1.54 Gbits/sec  393  1.95 MBytes
[ 7] 5.00-6.00 sec  188 MBytes  1.57 Gbits/sec    0  2.05 MBytes
[ 7] 6.00-7.00 sec  191 MBytes  1.60 Gbits/sec    0  2.12 MBytes
[ 7] 7.00-8.00 sec  200 MBytes  1.68 Gbits/sec    0  2.18 MBytes
[ 7] 8.00-9.00 sec  205 MBytes  1.72 Gbits/sec    0  2.22 MBytes
[ 7] 9.00-10.00 sec  204 MBytes  1.71 Gbits/sec    0  2.24 MBytes
[ ID] Interval      Transfer     Bitrate      Retr  sender receiver
[ 7] 0.00-10.00 sec  2.02 GBytes  1.73 Gbits/sec   663
[ 7] 0.00-10.02 sec  2.01 GBytes  1.72 Gbits/sec
iperf Done.
root@mininet-vm:/home/mininet#

```

Рис. 4.18: Запуск `iPerf3` на обоих хостах в разных режимах на обоих хостах

После завершения работы `iPerf3` на хосте `h1` остановим `iPerf3` на хосте `h2`, нажав `Ctrl + c`.

Удалим модифицированную конфигурацию на коммутаторе `s1` (рис. 4.19):

```

switch: s1 (root)@mininet-vm

s2-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether ca:22:34:fa:7d:85 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

s2-eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
ether de:79:bc:15:83:4e txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root tbf rate 10gb
it burst 5000000 limit 15000000
root@mininet-vm:/home/mininet# sudo tc qdisc del dev s1-eth2 root
root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 root handle 1: net
em delay 10ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev s1-eth2 parent 1: handle 2
: tbf rate 2gbit burst 1000000 limit 2000000
root@mininet-vm:/home/mininet# sudo tc qdisc del dev s1-eth2 root
root@mininet-vm:/home/mininet#

```

Рис. 4.19: Удаление модифицированной конфигурации на s1

4.3 Воспроизводимые эксперименты

Самостоятельно реализуйте воспроизводимые эксперименты по использованию TBF для ограничения пропускной способности. Постройте соответствующие графики.

Создадим каталог, где будем проводить наш эксперимент и перейдем в него (рис. 4.20):

```

mininet@mininet-vm:~$ mkdir -p ~/work/lab_netem_iii/simple-tbf
mininet@mininet-vm:~$ cd ~/work/lab_netem_iii/simple-tbf
mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$ nano lab_netem_iii.py
mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$

```

Рис. 4.20: Создание каталога

Создадим скрипт, реализующий наш воспроизводимый эксперимент (рис. 4.21):

```

GNU nano 4.8 lab_netem_iii.py
#!/usr/bin/env python
"""
Simple experiment.
Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI

from mininet.log import setLogLevel, info
import time

def emptyNet():

    "Create an empty network and add nodes to it."
    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s1 = net.addSwitch( 's1' )

    info( '*** Creating links\n' )
    net.addLink( h1, s1 )
    net.addLink( h2, s1 )

    info( '*** Starting network\n' )
    net.start()

    info( '*** Set rate\n' )

    h1.cmdPrint('tc qdisc add dev h1-eth0 root tbf rate 10gbit burst 5000000 limit 15000000')
    time.sleep(10) # Wait 10 seconds

    info( '*** Starting iperf server on h2\n' )
    h2.cmdPrint('iperf3 -s &')
    info( '*** Running iperf client from h1 to h2\n' )
    h1.cmdPrint('iperf3 -c ' + h2.IP() + ' | grep "MBytes" | awk \'{print $7}\'} > ping.dat')

    info( '*** Stopping network' )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()

```

Рис. 4.21: Скрипт lab_netem_iii.py

Создадим Makefile (рис. 4.22):

```

GNU nano 4.8 Makefile
all: ping.dat ping.png

ping.dat:
    sudo python lab_netem_iii.py
    sudo chown mininet:mininet ping.dat

ping.png: ping.dat
    ./ping_plot

clean:
    -rm -f *.dat *.png

```

Рис. 4.22: Создание Makefile

Создадим ping_plot (рис. 4.23):

```

GNU nano 4.8                                ping_plot
#!/usr/bin/gnuplot --persist
set terminal png crop
set output 'ping.png'
set xlabel "Packet number"
set ylabel "rate (Gbytes/sec)"
set grid
plot "ping.dat" with lines

```

Рис. 4.23: Создание ping_plot

Теперь запустим эксперимент и дождемся его окончания. Просмотрим содержимое нашего каталога после запуска эксперимента (рис. 4.24):

```

mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$ make
sudo python lab_netem_iii.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set rate
*** h1 : ('tc qdisc add dev h1-eth0 root tbf rate 10gbit burst 5000000 limit 15000000',)
*** Starting iperf server on h2
*** h2 : ('iperf3 -s &',)
*** Running iperf client from h1 to h2
*** h1 : ('iperf3 -c 10.0.0.2 | grep "MBytes" | awk '{print $7}' > ping.dat',)
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
./ping_plot
make: execvp: ./ping_plot: Permission denied
make: *** [Makefile:8: ping.png] Error 127
mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$ chmod +x ping_plot
mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$ make
./ping_plot
mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$ ls
lab_netem_iii.py Makefile ping.dat ping_plot ping.png
mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$

```

Рис. 4.24: Запуск скрипта и получение файлов

Для открытия, полученного графика воспользуемся geeqie (рис. 4.25):

```

mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$ ls
lab_netem_iii.py Makefile ping.dat ping_plot ping.png
mininet@mininet-vm:~/work/lab_netem_iii/simple-tbf$ geeqie ping.png

```

Рис. 4.25: Запуск geeqie

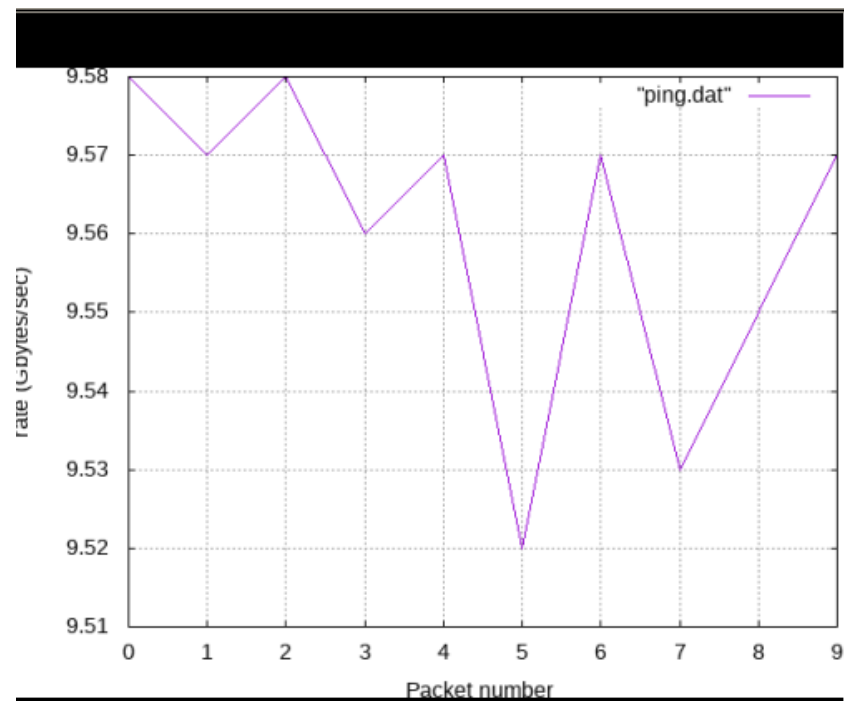


Рис. 4.26: Финальный график

5 Выводы

В результате выполнения данной лабораторной работы я познакомилась с принципами работы дисциплины очереди Token Bucket Filter, которая формирует входящий/исходящий трафик для ограничения пропускной способности, а также получила навыки моделирования и исследования поведения трафика посредством проведения интерактивного и воспроизводимого экспериментов в Mininet